

NECL (nec_lite) C-Core Spec Sheet

Time-series kernel for Cortex-M. Gleiche Flags wie heatshrink, lz4 und miniz. Der ehrliche Peer ist heatshrink — nicht lz4.

ARM Cortex-M0 / M3
malloc = 0 · Bound n+32
Block-RAM = O(n)
GCC 14.2.1 ELF-Map

2.020 KB Flash nec_lite Cortex-M0	2.236 KB Flash heatshrink W=8 Encoder+Decoder	O(n) Caller-RAM: scratch=n, dst=n+32	n + 32 Harte Obergrenze (heatshrink expandiert)
--------------------------------------	---	--	---

Unter gleichen arm-none-eabi-gcc -Os-Flags liegt nec_lite auf M0/M3 bei 1,9–2,0 KB Flash — in derselben Größenordnung wie heatshrink (2,17 / 2,24 KB Encoder+Decoder), klar unter lz4/miniz. Der Vorsprung ist nicht „der kleinste Codec im Segment“, sondern hartes n+32, Library-BSS = 0 und ein Delta-Frontend. heatshrink bleibt der bessere Allzweck-LZSS: konstantes Fenster-RAM, bessere Ratio, Streaming.

1. Flash — gleiche Harness, jetzt inkl. heatshrink

arm-none-eabi-gcc 14.2.1, -Os -mthumb -ffreestanding --gc-sections, 128-Byte-Harness, kein libc. heatshrink v0.4.1 statisch: W=8 L=4 USE_INDEX=1 DYNAMIC_ALLOC=0 (kanonische Small-MCU-Config). Flash = size -A .text. Quelle: mcu-size.json.

CPU	KERNEL	FLASH (.TEXT)	LIBRARY-BSS	HARNESS-BSS (N=128)
M3	nec_lite	1.896	0	420
M3	heatshrink enc+dec	2.168	1.856 †	548
M3	miniz tdefl/tinfl	4.116	0 ‡	516
M3	lz4 v1.10	4.308	0	404
M0	nec_lite	2.020	0	420
M0	heatshrink enc+dec	2.236	1.856 †	548
M0	miniz	4.828	0 ‡	516
M0	lz4	6.488	0	404

† heatshrink: Encoder-Fenster+Index 1.554 B + Decoder 302 B, konstant unabhängig von n. Der ~1 KB-Decoder-Claim ist Decoder-only (Objekt ~0,6 KB .text vor GC); hier zählt Encoder+Decoder, analog zu nec_lite. ‡ miniz tdefl_compress_mem_to_mem malloc't intern den Compressor (~zehn KB); MINIZ_NO_MALLOC macht diese API zur Laufzeit unbrauchbar — Flash-Zahl bleibt der Code. 420 B bei nec_lite ist src+dst+scratch für n=128, nicht „der Kernel braucht 420 B“.

2. RAM-Modell: Block vs. Stream

nec_lite, lz4, miniz — Block Die ganze Payload muss im RAM (oder XIP+RAM) liegen. nec_lite: schreibbar scratch = n plus dst = n+32 (≈ 2n+32), plus src falls nicht im Flash. Bei n = 16 KiB sind das ~32 KiB schreibbar — auf einem 2–4 KiB-SRAM-M0 der Engpass, nicht die 2 KB Flash.	heatshrink — Stream Sink/Poll in beliebigen Häppchen. Library-RAM = Encoder 1.554 B und/oder Decoder 302 B, unabhängig von der Dateigröße. IO-Puffer z. B. 32 B. Das ist der eigentliche RAM-Vorteil auf kleinem SRAM.
--	--

3. Ratio — Host, Roundtrip, dieselben Corpora

Quelle: `bash nec-core/scripts/codec_ratio.sh → ratio.txt.coded/orig.`

CORPUS	NEC_LITE	HEATSHRINK	LZ4	MINIZ
harness128 (i×3, FE_I16)	32,0%	112,5%	101,6%	108,6%
timeseries_i16 160 KB	100,0%	104,2%	100,4%	76,0%
ticks 144 KB	95,1%	51,2%	59,8%	40,1%
logline_16k (LZSS-freundlich)	100,2%	10,3%	0,6%	0,6%
random_16k	100,2%	112,3%	100,4%	100,1%

harness128 ist nec_lite-günstig (Delta+RLE). Auf Ticks und Logzeilen verliert RLE gegen LZSS/Deflate deutlich. heatshrink hat kein hartes n+32 — random_16k expandiert auf 112%.

4. Flash + komprimierte Nutzdaten (M0, 16 KiB Payload)

Gesamt = Codec-.text + coded. Das ist die Produktfrage, nicht Flash allein.

16 KIB CORPUS	NEC_LITE	HEATSHRINK	LZ4	MINIZ
ticks_16k	17.564	10.728	16.936	11.347
logline_16k	18.436	3.929	6.590	4.922
timeseries_16k	18.436	19.296	22.938	17.439
random_16k	18.436	20.629	22.938	21.223

Byte, kleiner ist besser. heatshrink/miniz gewinnen, sobald LZSS/Deflate greift. nec_lite gewinnt nur auf Incompressibility (Bound) und knapp auf AR(1)-int16 vs. heatshrink.

5. Was gilt / was nicht

Was gilt • Kein malloc im Kernel. Harte Grenze n+32. • Flash in heatshrink-Größenordnung, klar unter lz4/miniz. • Delta-Frontend kann auf kurzen int16-Läufen schlagen (harness128).	Was nicht gilt • Nicht kleiner als der Segment-Peer heatshrink in einem relevanten Sinn (11–14 % Flash, schlechtere Ratio, kein Stream). • 420 B RAM ist nicht konstant — skaliert mit n. • Nicht auf Silizium getestet. Timing/Energie offen.
--	--

6. Repro

```
bash nec-core/scripts/mcu_build.sh
MCU_CPU=cortex-m0 bash nec-core/scripts/mcu_build.sh
bash nec-core/scripts/codec_ratio.sh
```